AMS 530: Project 3, Problem 3.4

Matthew DiCicco

Problem setup and execution on SeaWulf

The work was initiated from the main login gateway of Stony Brook University's SeaWulf computing cluster. The MPI runs were executed using an interactive allocation on a compute node.

Request one compute node with 16 tasks:

```
salloc --partition=short-28core --nodes=1 --ntasks=16 --cpus-per-task=1
```

After the allocation is granted, enter an interactive shell and load the required modules:

```
srun --pty bash
module load mpich/gcc/3.4.2
module load anaconda/3-new
```

The MPICH module provides the MPI compiler and runtime. The Anaconda module provides Python and Matplotlib for plotting the benchmark results. If nested `srun` commands cannot launch from the interactive shell, execute the benchmark script from the original shell holding the allocation instead.

The `--mpi=pmi2` option in the benchmark script is essential for the MPI process launch used here. An earlier run without this option incorrectly identified the MPI process count. Adding it resolved the issue and produced the intended 1-, 2-, 4-, and 16-process measurements.

Compiling the C programs

```
gcc -O2 serial_small.c -o serial_small -lm
gcc -O2 serial_large.c -o serial_large -lm
mpicc -O2 parallel_large.c mesh.c jacobi.c redblack.c -o parallel_large -lm
```

The `-O2` option enables compiler optimization; `-lm` links the math library. The MPI program must be compiled with `mpicc`.

Running the single-process programs

```
./serial_small > serial_small_output.txt
./serial_large > serial_large_output.txt
```

The first program runs 10 iterations on the 4 × 4 interior mesh and prints both methods' residual after every iteration. The second runs 1,000 iterations on the 128 × 128 mesh, prints per-iteration timing readings, and ends with average iteration time, total time, and final residual for Jacobi, standard Gauss-Seidel, and red-black Gauss-Seidel.

Running the MPI benchmarks and making plots

```
bash run_benchmarks.sh
```

It runs the already compiled `parallel_large` program using:

```
srun --mpi=pmi2 -n 1 ./parallel_large
srun --mpi=pmi2 -n 2 ./parallel_large
srun --mpi=pmi2 -n 4 ./parallel_large
srun --mpi=pmi2 -n 16 ./parallel_large
```

The shell script collects the eight CSV records, creates a readable benchmark summary, and calls Python to generate both PDF plots. If the benchmarks have already finished, the plots can be regenerated **without rerunning MPI**:

```
python3 plot_results_fixed.py
```

Files and their purposes

The project files are organized into the following directories:

Location	Contents and purpose
`README_Project3_Final.md`	Project description and instructions for compiling and running the code.
`Serial_Small/`	Contains `serial_small.c`, the compiled `serial_small` executable, and `serial_small_output.txt`. This single-processor program builds the 16 × 16 system and performs 10 iterations of Jacobi and Gauss-Seidel on the 4 × 4 grid. The output records the residual at each iteration.
`Serial_Large/`	Contains `serial_large.c`, the compiled `serial_large` executable, and `serial_large_output.txt`. This single-processor program implements Jacobi, standard Gauss-Seidel, and checkerboard (red-black) Gauss-Seidel on the 128 × 128 grid. It performs 1,000 iterations of each method and records timing and residual results.
`MainCode&Results/`	Contains the MPI implementation of Jacobi and checkerboard (red-black) Gauss-Seidel, evaluated using 1, 2, 4, and 16 processors. It also contains the benchmark script, plotting script, compiled executable, and the `Results_/` directory.
`Superseded_Files/`	Stores earlier files that are not part of the current implementation.

The files in `MainCode&Results/` are:

File	Purpose
`mesh.h`	Shared MPI constants, indexing macro, and function declarations.
`mesh.c`	Sets up the right-hand side, exchanges neighboring halo rows, and computes the global residual.
`jacobi.c`	Parallel Jacobi update using a separate next-iterate array.
`redblack.c`	Parallel checkerboard (red-black) Gauss-Seidel update, alternating the two checkerboard colors.
`parallel_large.c`	MPI driver that initializes MPI, partitions rows, times the solvers, computes residuals, and prints CSV records.
`parallel_large`	Compiled MPI executable.
`run_benchmarks.sh`	Runs benchmarks with 1, 2, 4, and 16 MPI processes.
`plot_results_fixed.py`	Produces plots comparing measured and ideal speedup.
`Results_/`	Directory containing the benchmark results and plots.